

Agente Generativo de Planificación Académica o Personal

Materia: Introducción a la Programación con IA
Ciclo II - 2026
Ingeniería de Software y Negocios Digitales

Docente: Ing. Christopher Cañas

Este proyecto debe usar IA generativa como componente central de la solución.
La tecnología de desarrollo puede ser seleccionada por el grupo.

1. Descripción general del proyecto

El grupo desarrollará una aplicación que funcione como un agente generativo controlado para apoyar la planificación académica o personal. El sistema debe permitir registrar tareas, fechas, prioridades, disponibilidad y restricciones, para que la IA generativa proponga planes organizados, explique sus decisiones y ayude a replanificar cuando cambien las condiciones.

El proyecto no debe ser únicamente una lista de tareas con un botón de IA. Debe demostrar un flujo inteligente donde el sistema recopila información, envía contexto estructurado a un modelo generativo, recibe una propuesta de planificación, permite revisión humana y guarda trazabilidad de los cambios realizados.

2. Objetivo del proyecto

Construir una solución funcional que utilice IA generativa para crear, justificar y ajustar planes de trabajo académico o personal, considerando tareas pendientes, fechas límite, dificultad estimada, tiempo disponible, prioridades y restricciones definidas por el usuario.

3. Problema a resolver

Los estudiantes y profesionales suelen tener múltiples actividades pendientes, fechas límite, compromisos y prioridades que cambian durante la semana. Organizar manualmente estas tareas puede generar desorden, retrasos o planes poco realistas. Este proyecto propone una herramienta que use IA generativa para apoyar la organización, sin eliminar la responsabilidad del usuario de revisar, aprobar y ajustar el plan propuesto.

4. Alcance del sistema

4.1 Alcance funcional mínimo

- Permitir registrar tareas o actividades con título, descripción, categoría, fecha límite, prioridad, esfuerzo estimado y estado.
- Permitir registrar disponibilidad del usuario por día o por bloques de tiempo.
- Permitir definir restricciones, por ejemplo horarios ocupados, tareas que no pueden moverse, tiempo máximo por sesión o prioridad académica.
- Usar IA generativa para proponer un plan diario o semanal basado en tareas, disponibilidad y restricciones.
- Generar una justificación clara de por qué la IA ordenó las tareas de determinada forma.
- Permitir que el usuario acepte, edite, rechace o solicite una nueva versión del plan.
- Permitir replanificar cuando cambie una tarea, fecha límite, prioridad o disponibilidad.
- Mostrar alertas o recomendaciones cuando el plan sea poco realista, tenga sobrecarga o existan conflictos.
- Guardar historial de planes generados, cambios realizados, prompts, respuestas de IA, fecha y estado del plan.
- Permitir consultar tareas por estado: pendiente, en progreso, completada, atrasada o reprogramada.
- Incluir al menos tres escenarios de prueba: semana académica normal, semana con sobrecarga y semana con cambio inesperado.
- Incluir manejo de errores cuando la información sea insuficiente, la respuesta de la IA sea inválida o el plan generado no respete restricciones.

4.2 Alcance técnico mínimo

- La tecnología es libre: Python, Java, JavaScript, TypeScript, C#, PHP, Kotlin, Ruby u otra tecnología seleccionada por el estudiante.
- Debe existir una aplicación funcional: web, escritorio, API con frontend, aplicación móvil o solución equivalente.
- Debe existir persistencia de datos en base de datos, archivos estructurados o almacenamiento equivalente.
- Debe integrarse un modelo de IA generativa mediante API, SDK, herramienta local o proveedor externo.
- Debe existir una capa o módulo separado para la integración con IA generativa y otra para la lógica de planificación.
- Los prompts deben incluir contexto del usuario, lista de tareas, disponibilidad, restricciones, formato de salida y reglas para no ignorar fechas límite.
- El sistema debe validar que la respuesta de la IA tenga el formato esperado antes de mostrarla o guardarla como plan final.
- Las credenciales y API keys no deben estar quemadas directamente en el código fuente.
- El sistema debe dejar claro que la planificación generada es una recomendación y debe ser revisada por el usuario.

5. Requisitos funcionales obligatorios

Código	Requisito	Descripción
RF-01	Gestión de tareas	El sistema debe permitir registrar, editar, consultar y cambiar el estado de tareas o actividades.
RF-02	Disponibilidad	El usuario debe poder registrar bloques de tiempo disponibles o restricciones de agenda.
RF-03	Priorización	Cada tarea debe tener prioridad, fecha límite y esfuerzo estimado para alimentar la planificación.
RF-04	Plan generativo	La IA debe generar un plan diario o semanal con orden sugerido de tareas y tiempos recomendados.
RF-05	Justificación	La IA debe explicar las razones principales del orden, prioridad y distribución del plan.
RF-06	Revisión humana	El usuario debe poder aceptar, editar, rechazar o solicitar una nueva versión del plan.
RF-07	Replanificación	El sistema debe permitir generar un nuevo plan cuando cambien tareas, prioridades o disponibilidad.
RF-08	Detección de conflictos	El sistema debe indicar sobrecarga, fechas en riesgo, tareas sin tiempo disponible o información insuficiente.
RF-09	Historial	El sistema debe guardar planes generados, estado, fecha, cambios y observaciones del usuario.
RF-10	Trazabilidad de IA	El sistema debe guardar prompt, respuesta generada, modelo usado si aplica y versión del plan.
RF-11	Panel de seguimiento	El sistema debe mostrar avance básico: tareas pendientes, completadas, atrasadas y reprogramadas.
RF-12	Manejo de errores	El sistema debe manejar respuestas inválidas, fallos de IA, datos incompletos y conflictos no resolubles.

6. Requisitos no funcionales

Código	Categoría	Descripción
RNF-01	Arquitectura	El código debe estar organizado por responsabilidades: interfaz, tareas, disponibilidad, planificación, integración con IA, almacenamiento y validación.
RNF-02	Seguridad	No exponer API keys en el repositorio. Usar variables de entorno o configuración excluida del control de versiones.
RNF-03	Privacidad	No solicitar ni almacenar información sensible innecesaria. Los escenarios de prueba deben ser ficticios o académicos.
RNF-04	Validación	Validar fechas, prioridades, esfuerzo estimado, disponibilidad y estructura de la respuesta generada.
RNF-05	Trazabilidad	Registrar prompts, respuestas, fecha, versión del plan, ajustes del usuario y estado de aprobación.
RNF-06	Manejo de errores	Mostrar mensajes claros cuando falten datos, falle la IA o el plan no respete restricciones.
RNF-07	Usabilidad	La interfaz debe permitir entender y modificar el plan sin confusión.
RNF-08	Mantenibilidad	El proyecto debe poder instalarse y ejecutarse siguiendo instrucciones documentadas.

7. Uso obligatorio de IA generativa

El componente de IA generativa es obligatorio y debe ser parte central del proyecto. La IA debe analizar tareas, disponibilidad, restricciones y prioridades para generar un plan estructurado, explicar sus decisiones y proponer ajustes. No basta con ordenar tareas por fecha o prioridad usando reglas fijas.

Elemento	Requisito
Entrada mínima	Tareas, fechas límite, prioridad, esfuerzo estimado, disponibilidad, restricciones y preferencias de planificación.
Salida esperada	Plan diario o semanal, orden de tareas, tiempos sugeridos, justificación, riesgos, conflictos y recomendaciones.
Formato recomendado	JSON estructurado, tabla de planificación, calendario semanal o secciones claramente separadas.
Control humano	El usuario debe aprobar, corregir o rechazar el plan antes de considerarlo final.
Restricciones	La IA no debe ignorar fechas límite, disponibilidad ni restricciones declaradas. Si el plan no es posible, debe indicarlo claramente.
Evidencia	El grupo debe documentar prompts, respuestas de ejemplo, escenarios de prueba, replanificaciones y errores controlados.

8. Documentación obligatoria

- Documento técnico del proyecto desarrollado: arquitectura, tecnologías, instalación, configuración, estructura de carpetas e integración con IA.
- Documento funcional del proyecto desarrollado: descripción de pantallas, flujos, reglas de negocio y comportamiento esperado.
- Manual de usuario del proyecto desarrollado: instrucciones para registrar tareas, configurar disponibilidad, generar planes y replanificar.

- Catálogo de prompts: prompts utilizados, objetivo de cada prompt, versión, ejemplo de entrada y ejemplo de salida.
- Guía de ejecución: pasos para instalar, configurar variables de entorno y ejecutar el sistema.
- Evidencia de pruebas: escenarios académicos o personales, planes generados, replanificaciones, conflictos detectados y errores manejados.
- Muestra de resultados generados por la aplicación: al menos tres escenarios completos con plan inicial, explicación y ajuste posterior.

9. Criterios específicos de evaluación del proyecto

Criterio	Expectativa
Solución del problema	El sistema ayuda a organizar tareas y genera planes útiles, realistas y revisables.
Uso de IA generativa	La IA analiza contexto, genera planificación, justifica decisiones y apoya la replanificación.
Prompting	Los prompts controlan tareas, disponibilidad, restricciones, formato de salida y reglas para evitar planes irreales.
Calidad del agente	El sistema actúa como agente controlado: propone, explica y espera aprobación humana antes de aceptar cambios.
Arquitectura	El proyecto está organizado, separa responsabilidades y facilita cambios en reglas o criterios de planificación.
Seguridad y privacidad	El grupo protege credenciales y evita recopilar datos personales innecesarios.
Trazabilidad	Se registran prompts, respuestas, planes, versiones, ajustes humanos y estados de aprobación.
Defensa	Cada integrante explica su aporte y responde preguntas técnicas sobre flujo, prompts, IA, validaciones y seguridad.

10. Reglas y restricciones

- El proyecto debe ser desarrollado por el grupo y debe poder demostrarse durante la defensa.
- La IA generativa debe estar integrada en la aplicación, no solo usada para crear código o redactar documentación externa.
- No se permite presentar únicamente una presentación o mockup sin aplicación funcional.
- No se permite exponer credenciales, tokens o API keys en repositorios públicos.
- El grupo debe entregar los prompts utilizados. La ausencia de prompts afecta la calificación.
- Cada estudiante debe poder explicar una parte real del sistema y responder preguntas individuales.
- El agente no debe ejecutar acciones irreversibles sin aprobación del usuario.
- El sistema debe mostrar claramente cuando un plan no es posible o requiere ajustes.
- No se debe presentar la planificación generada como una decisión obligatoria; debe tratarse como recomendación revisable.

11. Recomendaciones de tecnología

El grupo puede elegir la tecnología con la que se sienta más cómodo. La evaluación no se centrará en el lenguaje elegido, sino en la calidad de la solución, arquitectura, integración con IA generativa, trazabilidad y defensa del proyecto.

Área	Opciones posibles
Frontend	React, Angular, Vue, Razor, Blazor, Flutter, mobile o interfaz de escritorio.
Backend	Python, Node.js, .NET, Java, PHP, Ruby u otra tecnología seleccionada por el estudiante.
Base de datos	PostgreSQL, SQL Server, MySQL, SQLite, MongoDB, Firebase o archivo estructurado justificado.
IA generativa	OpenAI, Azure OpenAI, Gemini, Claude, Llama, Mistral u otro modelo equivalente.

Área	Opciones posibles
Visualización	Lista priorizada, tablero Kanban, calendario semanal, tabla de seguimiento o dashboard simple.
Control de versiones	GitHub, GitLab, Azure DevOps o repositorio equivalente.

12. Fuera de alcance

- No se exige integración real con Google Calendar, Outlook u otras agendas externas.
- No se exige envío real de notificaciones por correo, SMS o WhatsApp.
- No se exige entrenamiento de modelos propios.
- No se exige optimización matemática avanzada de calendarios.
- No se exige despliegue en la nube, aunque puede agregarse como mejora.
- No se exige autenticación completa si el grupo justifica un flujo local o académico.
- No se exige que el sistema reemplace el criterio del usuario; la planificación debe ser una recomendación revisable.

13. Defensa del proyecto

Durante la defensa, el grupo debe demostrar el flujo completo: registro de tareas, configuración de disponibilidad, definición de restricciones, generación del plan con IA, justificación de la propuesta, edición o aprobación del usuario, replanificación ante un cambio, historial, trazabilidad de prompts y manejo de errores. Cada integrante deberá explicar su participación y responder preguntas individuales sobre el proyecto.

14. Resultado esperado

Al finalizar, el grupo debe entregar una solución funcional que evidencie el uso responsable y profesional de IA generativa para apoyar la planificación académica o personal. El proyecto debe demostrar criterio técnico, calidad de prompting, arquitectura básica, seguridad mínima, trazabilidad y capacidad de explicar las decisiones tomadas.